MÓDULO 2 · RESUMO VISUAL

SQL do Zero para quem já sabe Excel

Cada comando explicado com desenho, exemplo da mesma empresa da planilha, exercício e gabarito. Para praticar, use o Laboratório SQL: você digita e ele corrige na hora.

🟢 SELECT · FROM · WHERE
🟢 ORDER BY · GROUP BY · HAVING
🟠 JOINs · UNION
🔴 CASE · Subqueries · CTE · Window Functions

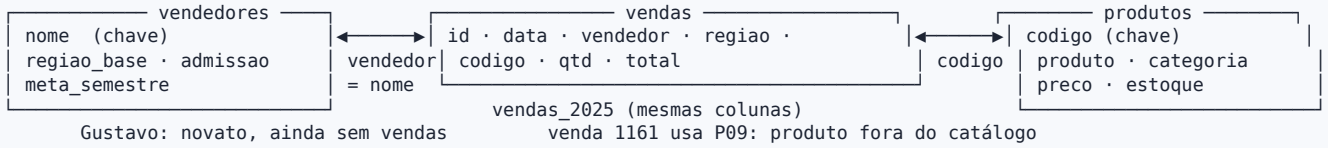
A ideia central em uma frase
SQL é uma forma de **pedir** dados a um banco escrevendo frases quase em inglês: "SELECIONE estas colunas DA tabela ONDE isto for verdade".

Como usar
1. Leia a página do bloco (5 a 10 min).
2. Abra o Laboratório SQL e faça os exercícios daquele bloco.
3. Errou? Leia a mensagem: ela diz se o problema é coluna, linha, valor ou ordem.
4. Revise o quadro "Decorar" nos dias 1, 3, 7 e 14.

🌉 A ponte: você já sabe SQL, só não sabia o nome

No Excel você faz…	No SQL você escreve…	O que acontece
Uma aba com dados (Tabela)	`FROM vendas`	Escolhe de qual tabela os dados vêm
Ocultar colunas que não interessam	`SELECT produto, preco`	Escolhe as colunas que aparecem
Autofiltro (Ctrl+Shift+L)	`WHERE regiao = 'Sul'`	Mantém só as linhas que passam na condição
Classificar de A a Z / maior para menor	`ORDER BY total DESC`	Ordena o resultado
Tabela Dinâmica / SOMASE	`GROUP BY regiao` + `SUM(total)`	Cria um grupo por região e resume cada um
Filtrar a Tabela Dinâmica pelo total	`HAVING SUM(total) > 60000`	Filtra os grupos depois de resumir
PROCV / PROCX entre duas abas	`JOIN produtos ON …`	Cola duas tabelas pela coluna-chave
Copiar uma tabela embaixo da outra	`UNION ALL`	Empilha resultados
SE(; ;)	`CASE WHEN … THEN … END`	Cria uma coluna com regra
Aba "Cálculos" de apoio	`WITH tabela AS ( … )` (CTE)	Etapa intermediária com nome
Ranking / =SOMA($B$2:B2) acumulado	`RANK() OVER` · `SUM() OVER`	Window functions: cálculo que "olha" outras linhas

🧩 O banco de treino (mesma empresa da planilha)



Esses dois "defeitos" são de propósito: eles fazem o LEFT JOIN e o RIGHT JOIN mostrarem diferença.

⚙️ A ordem em que você ESCREVE × a ordem em que o banco EXECUTA

Você escreve

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

🧠 Frase para decorar: **"Sempre Faço Waffles Gostosos Hoje, Ou Lanche"**.

O banco executa

FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Por isso o WHERE não enxerga o SUM (o grupo ainda não existe) e por isso o apelido do SELECT funciona no ORDER BY.

🟢 Fundamentos: SELECT, FROM, WHERE, ORDER BY

SELECT + FROM: "o quê" e "de onde"

```
SELECT produto, preco      ← quais COLUNAS
FROM   produtos;           ← de qual TABELA
                           ← ; termina a frase
 produtos                   resultado

┌──────┬────────┬──────┐   ┌────────┬──────┐
│codigo│produto │ preco│ → │produto │ preco│
│ P01  │Notebook│ 3500 │   │Notebook│ 3500 │
└──────┴────────┴──────┘   └────────┴──────┘
```

`SELECT *` = todas as colunas. Em empresa, evite `*` em tabelas grandes: traga só o que precisa.

WHERE: o autofiltro

```
SELECT id, total
FROM   vendas
WHERE  regiao = 'Sul'          ← texto: aspas SIMPLES
  AND  total  > 3000;          ← número: sem aspas

  linha passa?  regiao='Sul' ✓ E total>3000 ✓ → entra
                regiao='Sul' ✓ E total=640  ✗ → sai
```

Operadores: `= <> > < >= <=` · `AND` (E) · `OR` (OU) · `IN ('Sul','Norte')` · `BETWEEN 1000 AND 5000` · `LIKE 'Ma%'` (começa com Ma) · `IS NULL`

ORDER BY e LIMIT: ordenar e cortar

```
SELECT produto, preco
FROM   produtos
ORDER BY preco DESC      ← DESC: maior → menor
LIMIT 3;                  ← só as 3 primeiras
                          = um "TOP 3"
```

ASC (crescente) é o padrão. Dá para ordenar por 2 colunas: `ORDER BY regiao, total DESC`. No SQL Server o LIMIT se escreve `SELECT TOP 3 …`.

⚠️ Erros mais comuns

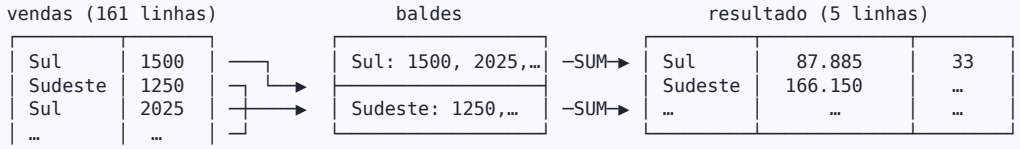
✗ `WHERE regiao = "Sul"` → use aspas simples `'Sul'`
✗ vírgula sobrando antes do FROM: `SELECT a, b, FROM`
✗ `WHERE total = NULL` → o certo é `IS NULL`
✗ misturar AND e OR sem parênteses:
`WHERE (regiao='Sul' OR regiao='Norte') AND total>3000`
✗ datas: no banco de treino são texto `'2026-03-01'` (ano-mês-dia)

Nº	Exercício (faça no Laboratório)	Gabarito
A	Produtos da categoria 'Papelaria'	`SELECT * FROM produtos WHERE categoria = 'Papelaria';`
B	Vendas do Norte OU Nordeste acima de 2000	`SELECT * FROM vendas WHERE regiao IN ('Norte','Nordeste') AND total > 2000;`
C	Os 2 produtos com MAIOR estoque	`SELECT produto, estoque FROM produtos ORDER BY estoque DESC LIMIT 2;`
D	Funcionários cujo nome começa com 'C'	`SELECT nome FROM funcionarios WHERE nome LIKE 'C%';`

🟢 Fundamentos: GROUP BY e HAVING (a Tabela Dinâmica do SQL)

GROUP BY: separar em baldes e resumir cada balde

```
SELECT regiao, SUM(total) AS faturamento, COUNT(*) AS vendas
FROM   vendas
GROUP BY regiao;
```



Funções de agregação: `SUM` (SOMA) · `COUNT(*)` (conta linhas) · `COUNT(coluna)` (conta não vazios) · `AVG` (MÉDIA) · `MIN` · `MAX` · `COUNT(DISTINCT vendedor)` (conta sem repetir). **AS** dá um apelido à coluna.

WHERE × HAVING (o que mais confunde)

```
SELECT vendedor, SUM(total)
FROM   vendas
WHERE  regiao = 'Sul'          ← 1º filtra LINHAS
GROUP BY vendedor              ← 2º agrupa
HAVING SUM(total) > 20000;     ← 3º filtra GRUPOS

WHERE  = peneira ANTES de juntar
HAVING = peneira DEPOIS de juntar
```

Regra de ouro do GROUP BY

Toda coluna do SELECT que **não** está dentro de SUM/COUNT/AVG... precisa estar no GROUP BY.

✗ `SELECT vendedor, regiao, SUM(total) … GROUP BY vendedor` (qual região mostrar, se o Bruno vendeu em várias?)

✓ `… GROUP BY vendedor, regiao`

✗ `WHERE SUM(total) > 1000` → erro "misuse of aggregate". Use HAVING.

Nº	Exercício	Gabarito
E	Quantidade vendida (soma de qtd) por código de produto	`SELECT codigo, SUM(qtd) FROM vendas GROUP BY codigo;`
F	Ticket médio por região, do maior para o menor	`SELECT regiao, AVG(total) AS ticket FROM vendas GROUP BY regiao ORDER BY ticket DESC;`
G	Regiões com mais de 30 vendas	`SELECT regiao, COUNT(*) FROM vendas GROUP BY regiao HAVING COUNT(*) > 30;`
H	Salário médio por departamento (só os acima de 7000)	`SELECT departamento, AVG(salario) FROM funcionarios GROUP BY departamento HAVING AVG(salario) > 7000;`

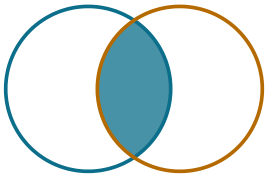
📌 Decorar
Ordem de escrita: S-F-W-G-H-O-L. Texto em aspas simples. Filtro de soma = HAVING.

💡 Entender
GROUP BY transforma muitas linhas em uma linha por grupo. Depois dele, só existe o que foi agrupado ou resumido.

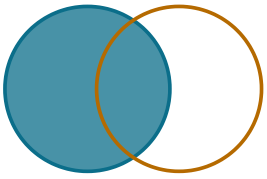
🏢 Cai na empresa
"Faturamento por mês/região/vendedor" e "quem passou da meta": GROUP BY + HAVING em toda entrevista.

🟠 Intermediário: JOINs (o PROCX do SQL)

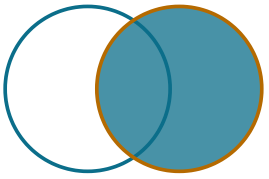
Um JOIN cola duas tabelas lado a lado usando uma coluna que existe nas duas (a **chave**). Você escreve `ON` para dizer qual coluna de cá combina com qual de lá.



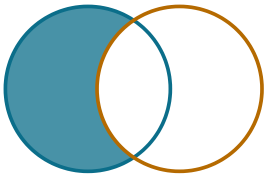
INNER JOIN
só quem tem par



LEFT JOIN
tudo da esquerda



RIGHT JOIN
tudo da direita



LEFT + IS NULL
só quem NÃO tem par

INNER JOIN: só as combinações que existem

```
SELECT v.id, p.produto, p.categoria
FROM   vendas v                  ← v = apelido
JOIN   produtos p                ← p = apelido
  ON   v.codigo = p.codigo;      ← a chave

 vendas.codigo   produtos.codigo
     P05  ──────────►  P05 Mesa      ✓ entra
     P09  ─── ✗ (não existe)         ✗ some!
```

`JOIN` sozinho = `INNER JOIN`.

LEFT JOIN: não perca ninguém da esquerda

```
SELECT d.nome, COUNT(v.id) AS vendas
FROM   vendedores d              ← esquerda: TODOS ficam
LEFT JOIN vendas v
  ON   v.vendedor = d.nome
GROUP BY d.nome;

 Ana      ✓ 25 vendas
 …
 Gustavo  ✗ sem par → colunas de vendas = NULL → 0
```

Use `COUNT(v.id)`: o `COUNT(*)` contaria a linha vazia como 1.

RIGHT JOIN = LEFT JOIN espelhado

```
FROM produtos p RIGHT JOIN vendas v ON …
   é o mesmo que
FROM vendas v   LEFT JOIN produtos p ON …
```

Na prática quase todo mundo usa LEFT e troca a ordem das tabelas. Saber os dois cai em entrevista.

UNION / UNION ALL: empilhar

```
SELECT id, data, total FROM vendas_2025
UNION ALL                        ← mantém repetidos
SELECT id, data, total FROM vendas;

  ┌ 2025 ┐
  └──────┘     mesmas colunas,
  ┌ 2026 ┐     mesma ordem,
  └──────┘     mesmos tipos
```

`UNION` sem ALL remove linhas repetidas (como o ÚNICO). JOIN = lado a lado; UNION = um embaixo do outro.

Erros comuns: esquecer o ON (vira um "cruzamento" de todas as linhas com todas) · coluna ambígua: `codigo` existe nas duas tabelas, escreva `v.codigo` · filtrar a tabela da direita no WHERE depois de um LEFT JOIN (transforma em INNER sem querer; coloque a condição no ON).

Nº	Exercício	Gabarito
I	Faturamento por categoria e região	`SELECT p.categoria, v.regiao, SUM(v.total) FROM vendas v JOIN produtos p ON v.codigo = p.codigo GROUP BY p.categoria, v.regiao;`
J	Produtos que nunca foram vendidos	`SELECT p.produto FROM produtos p LEFT JOIN vendas v ON v.codigo = p.codigo WHERE v.id IS NULL;` (resultado: nenhum, todos venderam)
K	% da meta de cada vendedor	`SELECT d.nome, SUM(v.total) / d.meta_semestre FROM vendedores d LEFT JOIN vendas v ON v.vendedor = d.nome GROUP BY d.nome, d.meta_semestre;`

🔴 Avançado: CASE e Subqueries

CASE: o SE do SQL

```
SELECT id, total,
  CASE
    WHEN total >= 5000 THEN 'Grande'   ← testa 1º
    WHEN total >= 1000 THEN 'Média'    ← depois este
    ELSE 'Pequena'                       ← senão
  END AS faixa
FROM vendas;

 total 7000 → 1º WHEN ✓ → 'Grande' (para aqui)
 total 1250 → 1º ✗ → 2º ✓ → 'Média'
```

Uso clássico: `SUM(CASE WHEN regiao='Sul' THEN total ELSE 0 END)` cria colunas tipo Tabela Dinâmica (Sul, Norte… lado a lado).

Subquery: uma consulta dentro da outra

```
SELECT id, total
FROM   vendas
WHERE  total > ( SELECT AVG(total) FROM vendas );
                └──── roda PRIMEIRO ────┘
                       devolve 2467,05
        → vira: WHERE total > 2467.05

WHERE codigo IN (SELECT codigo FROM produtos
                  WHERE categoria = 'Móveis')
                → devolve a LISTA ('P04','P05')
```

Subquery que devolve **1 valor** → use com `= > <`. Que devolve **lista** → use com `IN`.

🔴 Avançado: CTE (WITH) e Window Functions

CTE: dê nome a uma etapa (a aba "Cálculos")

```
WITH tot AS (                        ← etapa 1 com nome
  SELECT vendedor, SUM(total) AS fat
  FROM vendas GROUP BY vendedor
)
SELECT vendedor, fat                 ← etapa 2 usa a 1
FROM   tot
WHERE  fat > (SELECT AVG(fat) FROM tot);
```

Mesma coisa que subquery, mas lida de cima para baixo como uma receita. Várias etapas: `WITH a AS (…), b AS (…) SELECT …`

Window Functions: calcular SEM perder as linhas

```
GROUP BY:  161 linhas → 6 linhas (junta)
OVER():    161 linhas → 161 linhas + coluna nova

função() OVER ( PARTITION BY regiao   ← reinicia por grupo
                ORDER BY total DESC ) ← ordem dentro dele

 regiao  total  RANK
 Sul     10500   1  ┐
 Sul     10500   1  │ janela "Sul" (empate!)
 Sul      7000   3  ┘
 Norte   10500   1  ← recomeça
```

Window function	O que faz	Uso na empresa
`ROW_NUMBER()`	Numera 1, 2, 3… sem empate	Pegar o último pedido de cada cliente (WHERE n = 1)
`RANK()` · `DENSE_RANK()`	Ranking com empate (1,1,3 · 1,1,2)	Ranking de vendedores, top produto por região
`SUM(x) OVER (ORDER BY mes)`	Soma acumulada	Faturamento acumulado do ano
`LAG(x)` · `LEAD(x)`	Valor da linha anterior · da próxima	Crescimento mês a mês
`AVG(x) OVER (PARTITION BY regiao)`	Média do grupo ao lado de cada linha	"Esta venda está acima da média da região?"

Erro clássico: filtrar window function no WHERE da mesma consulta (`WHERE RANK() … = 1`). Não pode, porque o WHERE roda antes. Calcule numa CTE e filtre fora dela.

📌 Decorar
CASE WHEN THEN ELSE END. OVER = janela. PARTITION BY = "por grupo". LAG = linha de cima.

💡 Entender
GROUP BY encolhe a tabela; window function mantém todas as linhas e só adiciona uma coluna.

🏢 Cai na empresa
CTE + ROW_NUMBER para deduplicar, LAG para variação %, CASE para faixas. Top 3 de entrevista de analista.

✍️ Os 27 exercícios do Laboratório (tente antes de olhar o gabarito)

Os mesmos do Laboratório SQL, na mesma ordem. Lá a correção é automática.

Nº	Tema	Enunciado
1	FUNDA SELECT *	Mostre TODAS as colunas e linhas da tabela produtos.
2	FUNDA SELECT colunas	Mostre só o nome do produto e o preço.
3	FUNDA WHERE	Liste produto e preço dos produtos que custam MAIS de 500.
4	FUNDA WHERE + AND	Liste o id e o total das vendas da região 'Sul' com total acima de 3000.
5	FUNDA ORDER BY	Liste produto e preço do MAIS caro para o MAIS barato.
6	FUNDA ORDER BY + LIMIT	Mostre o nome e o preço dos 3 produtos MAIS BARATOS.
7	FUNDA WHERE com datas	Liste id, data e total das vendas de MARÇO/2026.
8	FUNDA COUNT / SUM / AVG	Em UMA linha: quantidade de vendas, soma do total e média do total (tabela vendas).
9	FUNDA GROUP BY	Mostre o faturamento (soma do total) por região.
10	FUNDA GROUP BY + COUNT	Quantas vendas cada vendedor fez? Ordene do que mais vendeu para o que menos vendeu.
11	FUNDA HAVING	Mostre os vendedores cujo faturamento passou de 60000 (vendedor e soma).
12	INTER INNER JOIN	Liste id da venda, nome do produto e categoria (junte vendas com produtos pelo código).
13	INTER JOIN + GROUP BY	Faturamento por CATEGORIA (a categoria está na tabela produtos).
14	INTER LEFT JOIN	Liste TODOS os vendedores (tabela vendedores) e quantas vendas cada um fez, inclusive quem fez zero.
15	INTER LEFT JOIN + IS NULL	Quais vendedores AINDA NÃO fizeram nenhuma venda? (só o nome)
16	INTER RIGHT JOIN	Liste id da venda e nome do produto mantendo TODAS as vendas, mesmo as de produto fora do catálogo.
17	INTER UNION ALL	Empilhe id, data e total das vendas de 2025 (vendas_2025) com as de 2026 (vendas).
18	INTER UNION	Liste as regiões que aparecem em vendas_2025 OU em vendas, sem repetir.
19	AVANÇ CASE	Para cada venda mostre id, total e a faixa: 'Grande' (>= 5000), 'Média' (>= 1000) ou 'Pequena'.
20	AVANÇ CASE + GROUP BY	Quantas vendas há em cada faixa (Grande / Média / Pequena)?
21	AVANÇ Subquery	Liste id e total das vendas ACIMA da média geral de vendas.
22	AVANÇ Subquery com IN	Liste id e total das vendas de produtos da categoria 'Móveis' (sem usar JOIN).
23	AVANÇ CTE (WITH)	Usando WITH, calcule o faturamento por vendedor e mostre só quem ficou ACIMA da média dos vendedores.
24	AVANÇ ROW_NUMBER	Faça o ranking dos vendedores por faturamento: vendedor, faturamento e posição (1 = maior).
25	AVANÇ SUM OVER (acumulado)	Mostre o mês (AAAA-MM), o faturamento do mês e o ACUMULADO do ano.
26	AVANÇ PARTITION BY	Para cada venda mostre id, região, total e a posição dela DENTRO da sua região (RANK, maior = 1). Só as posições 1 de cada região.
27	AVANÇ LAG	Mostre mês, faturamento e o faturamento do mês ANTERIOR (para comparar crescimento).

✅ Gabarito comentado

Nº	Consulta	Por que funciona
1	`SELECT *` `FROM produtos;`	O * significa 'todas as colunas'. FROM diz de qual tabela.
2	`SELECT produto, preco` `FROM produtos;`	Escolher colunas = esconder as colunas que não interessam, como ocultar colunas no Excel.

3	`SELECT produto, preco`
	`FROM produtos`
	`WHERE preco > 500;`
		WHERE é o filtro: só passam as linhas em que a condição é verdadeira.

4	`SELECT id, total`
	`FROM vendas`
	`WHERE regiao = 'Sul' AND total > 3000;`
		AND = as duas condições ao mesmo tempo (igual ao E do Excel).

5	`SELECT produto, preco`
	`FROM produtos`
	`ORDER BY preco DESC;`
		ORDER BY ordena. ASC = crescente (padrão), DESC = decrescente.

6	`SELECT produto, preco`
	`FROM produtos`
	`ORDER BY preco`
	`LIMIT 3;`
		LIMIT pega só as primeiras linhas. Ordenar antes é o que transforma em 'top 3'.

7	`SELECT id, data, total`
	`FROM vendas`
	`WHERE data BETWEEN '2026-03-01' AND '2026-03-31';`
		BETWEEN inclui as duas pontas. É o SOMASES com data >= e <= do Excel.

8	`SELECT COUNT(*), SUM(total), AVG(total)` `FROM vendas;`	Funções de agregação resumem várias linhas em uma só: CONT.VALORES, SOMA e MÉDIA do Excel.

9	`SELECT regiao, SUM(total) AS faturamento`
	`FROM vendas`
	`GROUP BY regiao;`
		GROUP BY cria um 'balde' por região e soma cada balde. É a Tabela Dinâmica do SQL.

10	`SELECT vendedor, COUNT(*) AS qtd_vendas`
	`FROM vendas`
	`GROUP BY vendedor`
	`ORDER BY qtd_vendas DESC, vendedor;`
		Agrupa por vendedor, conta as linhas de cada grupo e ordena pelo resultado.

11	`SELECT vendedor, SUM(total) AS faturamento`
	`FROM vendas`
	`GROUP BY vendedor`
	`HAVING SUM(total) > 60000;`
		WHERE filtra LINHAS antes de agrupar. HAVING filtra GRUPOS depois de agrupar.

12	`SELECT v.id, p.produto, p.categoria`
	`FROM vendas v`
	`INNER JOIN produtos p ON v.codigo = p.codigo;`
		INNER JOIN cola as tabelas onde o código bate nos dois lados. A venda do P09 (fora do catálogo) some.

13	`SELECT p.categoria, SUM(v.total) AS faturamento`
	`FROM vendas v`
	`JOIN produtos p ON v.codigo = p.codigo`
	`GROUP BY p.categoria;`
		É o PROCX + SOMASE do Excel em uma única consulta.

14	`SELECT d.nome, COUNT(v.id) AS qtd_vendas`
	`FROM vendedores d`
	`LEFT JOIN vendas v ON v.vendedor = d.nome`
	`GROUP BY d.nome;`
		LEFT JOIN mantém TODAS as linhas da esquerda. Gustavo aparece com 0 porque COUNT(v.id) ignora os vazios (NULL).

15	`SELECT d.nome`
	`FROM vendedores d`
	`LEFT JOIN vendas v ON v.vendedor = d.nome`
	`WHERE v.id IS NULL;`
		Quem não tem par do lado direito fica com NULL. Filtrar IS NULL acha os 'sem par'. Truque muito usado.

16	`SELECT v.id, p.produto`
	`FROM vendas v`
	`LEFT JOIN produtos p ON v.codigo = p.codigo;`
		RIGHT JOIN = LEFT JOIN com as tabelas trocadas de lugar. A venda 1161 (P09) aparece com produto NULL.

17	`SELECT id, data, total`
	`FROM vendas_2025`
	`UNION ALL SELECT id, data, total`
	`FROM vendas;`
		UNION empilha uma tabela embaixo da outra. As colunas precisam ser as mesmas, na mesma ordem.

18	`SELECT regiao`
	`FROM vendas_2025`
	`UNION SELECT regiao`
	`FROM vendas;`
		UNION ALL mantém repetidos (mais rápido). UNION remove repetidos (como o ÚNICO do Excel).

19	`SELECT id, total, CASE WHEN total >= 5000 THEN 'Grande'`
	`WHEN total >= 1000 THEN 'Média' ELSE 'Pequena' END AS faixa`
	`FROM vendas;`
		CASE é o SE do SQL. Ele testa de cima para baixo e para no primeiro WHEN verdadeiro.

20	`SELECT CASE WHEN total >= 5000 THEN 'Grande' WHEN total >= 1000 THEN 'Média' ELSE 'Pequena' END AS faixa, COUNT(*) AS qtd`
	`FROM vendas`
	`GROUP BY faixa;`
		Você cria uma categoria nova na hora e já resume por ela.

21	`SELECT id, total`
	`FROM vendas`
	`WHERE total > (SELECT AVG(total)`
	`FROM vendas);`
		A consulta de dentro roda primeiro e devolve um número (a média). A de fora usa esse número.

22	`SELECT id, total`
	`FROM vendas`
	`WHERE codigo IN (SELECT codigo`
	`FROM produtos`
	`WHERE categoria = 'Móveis');`
		A subquery devolve uma LISTA de códigos; o IN pergunta 'está nessa lista?'.

23	`WITH tot AS (SELECT vendedor, SUM(total) AS fat`
	`FROM vendas`
	`GROUP BY vendedor)`
	`SELECT vendedor, fat`
	`FROM tot`
	`WHERE fat > (SELECT AVG(fat)`
	`FROM tot);`
		CTE é uma 'tabela temporária com nome', como uma aba de Cálculos. Deixa a consulta legível em etapas.

24	`SELECT vendedor, SUM(total) AS fat, ROW_NUMBER() OVER (ORDER BY SUM(total) DESC) AS posicao`
	`FROM vendas`
	`GROUP BY vendedor`
	`ORDER BY posicao;`
		Window functions calculam 'olhando a janela' de linhas sem juntar tudo numa só. ROW_NUMBER numera em ordem.

25	`SELECT substr(data,1,7) AS mes, SUM(total) AS fat, SUM(SUM(total)) OVER (ORDER BY substr(data,1,7)) AS acumulado`
	`FROM vendas`
	`GROUP BY mes`
	`ORDER BY mes;`
		O OVER (ORDER BY mes) vai somando linha a linha, como =SOMA($B$2:B2) arrastado no Excel.

26	`WITH r AS (SELECT id, regiao, total, RANK() OVER (PARTITION BY regiao`
	`ORDER BY total DESC) AS posicao`
	`FROM vendas)`
	`SELECT id, regiao, total, posicao`
	`FROM r`
	`WHERE posicao = 1;`
		PARTITION BY reinicia o ranking a cada região (é o 'GROUP BY' das window functions). Não dá para filtrar window function no WHERE da mesma consulta, por isso a CTE.

27	`SELECT substr(data,1,7) AS mes, SUM(total) AS fat, LAG(SUM(total)) OVER (ORDER BY substr(data,1,7)) AS mes_anterior`
	`FROM vendas`
	`GROUP BY mes`
	`ORDER BY mes;`
		LAG 'espia' a linha de cima. Com ele dá para calcular variação: (fat - mes_anterior) / mes_anterior.

🧠 Cola de bolso + plano de memorização

```
SELECT   colunas, SUM(x) AS apelido          -- o quê
FROM     tabela t                             -- de onde
JOIN     outra o ON t.chave = o.chave         -- colar tabelas
WHERE    condição de LINHA                    -- filtro antes
GROUP BY colunas não resumidas                -- baldes
HAVING   condição com SUM/COUNT               -- filtro depois
ORDER BY coluna DESC                          -- ordenar
LIMIT    10;                                  -- cortar
```

📅 Repetição espaçada para SQL

Quando	O que fazer (15 a 20 min)
Dia 0	Exercícios 1 a 11 no Laboratório
Dia 1	Refaça 5, 9 e 11 sem olhar + exercícios 12 a 18
Dia 3	Escreva a "cola de bolso" de memória num papel
Dia 7	Exercícios 19 a 27
Dia 14	Modo livre: refaça o Dashboard de Vendas do Excel só com SQL
Dia 30	Simulado (peça no chat)

🎨 Âncoras visuais

🧺 **GROUP BY** = separar roupa em cestos por cor
🚪 **WHERE** = segurança na porta (antes de entrar) · **HAVING** = segurança na saída
🔗 **JOIN** = PROCX entre tabelas · **ON** = "pela coluna..."
🫂 **LEFT JOIN** = "ninguém da esquerda fica de fora"
🥞 **UNION** = panquecas empilhadas
🪟 **OVER** = olhar pela janela as linhas vizinhas
📒 **CTE** = rascunho com nome antes da resposta final

🗣️ Teste Feynman (responda em voz alta)

1. Por que não dá para usar SUM no WHERE? · 2. O que o LEFT JOIN faz com o Gustavo? · 3. Qual a diferença entre UNION e UNION ALL? · 4. Em que GROUP BY e OVER são diferentes? · 5. Quando uma subquery usa IN em vez de = ?

Travou em alguma? Releia só aquele quadro e volte ao Laboratório.

Onde o SQL muda entre bancos

Tudo deste guia funciona igual em PostgreSQL, MySQL, SQL Server e BigQuery, com três diferenças comuns: **LIMIT** (no SQL Server é `SELECT TOP 10`), **datas** (`substr(data,1,7)` do SQLite vira `FORMAT(data,'yyyy-MM')` no SQL Server ou `DATE_TRUNC('month', data)` no PostgreSQL) e **aspas em nomes** (colchetes [ ] no SQL Server).